ASSIGNMENT #5 IMPLEMENTING THE DATABASE OF THE SYSTEM USING SQL SERVERCOURSE: SYSTEM DEVELOPMENT COURSE CODE: 420-940-VA WEEK: 8

INTRODUCTION

In Week 8 you learned how to move your project from temporary in-memory storage (List<T>) to a real persistent database using SQL Server and ADO.NET. This assignment requires you to apply those skills to your own semester project by implementing a working database backend that replaces the temporary storage used in earlier weeks.

ASSIGNMENT OBJECTIVES

By completing this assignment, you will be able to:

- Create a real SQL Server database and tables that match your Week 6 design
- Establish a secure connection between your C# Windows Forms application and SQL Server
- Implement full CRUD (Create, Read, Update, Delete) operations using parameterized SQL statements and ADO.NET
- Load and display data in a DataGridView from the database
- Follow basic DevOps practices for configuration and file organization
- Document your work professionally and reflect on the transition from temporary to persistent storage

REQUIRED WORK

You must complete **all** of the following parts:

PART 1: DATABASE CREATION (SQL SERVER)

Create a new database in SQL Server (suggested name: YourProjectDB). Create all required tables with:

- Appropriate data types
- Primary Keys (using IDENTITY)
- Foreign Keys with referential integrity
- NOT NULL constraints where appropriate

Run the table-creation script and insert seed data for testing.

PART 2: CONNECTION SETUP

Add a connection string to your project in App.config. Create a button or method that tests the connection and shows a success or error message.

PART 3: FULL CRUD IMPLEMENTATION

Replace your previous in-memory List<T> CRUD operations with real SQL operations in your Windows Forms application:

- **Create**: Insert new records using parameterized INSERT
- **Read**: Load records into a DataGridView using SELECT with JOINs where needed
- **Update**: Modify existing records using parameterized UPDATE

- **Delete**: Remove records using parameterized DELETE

Include the optional helper methods from Lab 8 (ClearFields() and dgv_CellClick) if they improve usability.

PART 4: DEVOPS & CONFIGURATION NOTE

Create a short document or section (1/2 page) that shows:

- Where the connection string is stored
- Folder structure for SQL scripts
- Any configuration best practices you followed

PART 5: PERSONAL JOURNAL 4

Write a short reflection (1/2 to 1 page) answering:

- What was the biggest technical challenge when moving from List<T> to SQL Server?
- How does the database change the realism and reliability of your project?
- What did you learn about DevOps and deployment from this week?

Submission Requirements

Upload **one** compressed file to Omnivox: Assignment5_Database_YourLastName.zip

The zip file must contain:

1. The complete Visual Studio project folder (including all code and forms)
2. The SQL script file(s) used to create tables and seed data
3. A Word document named Assignment5_Database_YourLastName.docx that contains:
    - All sections from Part 1 to Part 5 above
    - Screenshots of your database tables, successful connection test, and DataGridView
    - Personal Journal 4

LATE POLICY

Work must be submitted on time through Omnivox. Late submissions lose 10% per day and are not accepted after three calendar days.

Evaluation The instructor will evaluate:

- Correctness and completeness of the database tables and relationships
- Quality and safety of the ADO.NET CRUD code (parameterized queries, error handling)
- Successful connection and DataGridView functionality
- Clarity of documentation and screenshots
- Thoughtfulness of Personal Journal 4

Weight: As announced by the instructor in class or on Omnivox, according to the course evaluation structure.

DETAILED ANSWERS FOR ALL PARTS, BUT SHOULDBE VERIFIED AND COMPLETED IF REQUIRED:

Part 1: Database Creation

SQL

```
CREATE DATABASE MyProjectDB;

-- Creates the actual database that will hold all project data permanently.


GO


USE MyProjectDB;

-- Switches the current context to the new database so all tables are created inside it.


GO


CREATE TABLE Students

(

   studentID INT IDENTITY(1,1) PRIMARY KEY,

   -- Auto-incrementing primary key that uniquely identifies each student.

   name NVARCHAR(100) NOT NULL,

   -- Required Unicode text field for student name.

   email NVARCHAR(150) NOT NULL,

   program NVARCHAR(100) NOT NULL
```

```
);

CREATE TABLE Advisors
(
    advisorID INT IDENTITY(1,1) PRIMARY KEY,
    -- Auto-incrementing primary key for each advisor.
    name NVARCHAR(100) NOT NULL,
    department NVARCHAR(100) NOT NULL
);

CREATE TABLE Appointments
(
    appointmentID INT IDENTITY(1,1) PRIMARY KEY,
    -- Auto-incrementing primary key for each appointment.
    studentID INT NOT NULL,
    advisorID INT NOT NULL,
    [date] DATETIME NOT NULL,
    -- Square brackets used because "date" is a reserved word.
    reason NVARCHAR(255) NOT NULL,
    status NVARCHAR(50) NOT NULL,

    CONSTRAINT FK_Appointments_Students
        FOREIGN KEY (studentID) REFERENCES Students(studentID),
    CONSTRAINT FK_Appointments_Advisors
        FOREIGN KEY (advisorID) REFERENCES Advisors(advisorID)
);
```

Seed data (run after the tables are created)

SQL

```
INSERT INTO Students (name, email, program)
VALUES
('Pejman Azadi', 'pejman@example.com', 'Computer Science'),
('Sara Lee', 'sara@example.com', 'Software Engineering');

INSERT INTO Advisors (name, department)
VALUES
('Dr. Chen', 'Computer Science'),
('Dr. Smith', 'Engineering');
```

Part 2: Connection Setup

C#

```
using System.Configuration;
using System.Data.SqlClient;

private string connectionString =
    ConfigurationManager.ConnectionStrings["MyProjectDB"].ConnectionString;
// Reads the connection string from App.config so it is stored in one central place.

private void btnTestConnection_Click(object sender, EventArgs e)
{
    try
    {
        using (SqlConnection conn = new SqlConnection(connectionString))
        // Creates a new SqlConnection object using the connection string from App.config.
        // The using statement ensures the connection is closed and disposed of automatically.
        {
```

```
            conn.Open();
            // Opens the actual connection to SQL Server.
            // If the connection fails, the program jumps to the catch block.

            MessageBox.Show("Database connection successful.");
            // Shows a message box to confirm that the application can talk to SQL Server.
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show("Database error: " + ex.Message);
        // Displays the error message returned by the system so the student can troubleshoot the problem.
    }
}
```

Part 3: Full CRUD Implementation (complete answer)

C#

```
using System.Data;
using System.Data.SqlClient;
using System.Configuration;

// Read
private void LoadAppointments()
// Declares a method named LoadAppointments that will retrieve rows from SQL Server and display them in the DataGridView.
{
    string query = "SELECT a.appointmentID, s.name AS StudentName, adv.name AS AdvisorName, a.[date], a.reason, a.status " +
                   "FROM Appointments a " +
```

```
            "INNER JOIN Students s ON a.studentID = s.studentID " +
            "INNER JOIN Advisors adv ON a.advisorID = adv.advisorID " +
            "ORDER BY a.[date] ASC";

    using (SqlDataAdapter adapter = new SqlDataAdapter(query, connectionString))
    {
        DataTable dt = new DataTable();
        adapter.Fill(dt);
        dgvAppointments.DataSource = dt;
    }
}

// Create
private void btnCreate_Click(object sender, EventArgs e)
{
    if (txtStudentID.Text == "" || txtAdvisorID.Text == "" || txtReason.Text == "")
    {
        MessageBox.Show("Please complete all required fields.");
        return;
    }

    if (!int.TryParse(txtStudentID.Text, out int studentID))
    {
        MessageBox.Show("Student ID must be a valid integer.");
        return;
    }

    if (!int.TryParse(txtAdvisorID.Text, out int advisorID))
```

```
    {
        MessageBox.Show("Advisor ID must be a valid integer.");
        return;
    }

    string query = "INSERT INTO Appointments (studentID, advisorID, [date], reason, status) " +
                   "VALUES (@studentID, @advisorID, @date, @reason, @status)";

    using (SqlConnection conn = new SqlConnection(connectionString))
    using (SqlCommand cmd = new SqlCommand(query, conn))
    {
        conn.Open();
        cmd.Parameters.AddWithValue("@studentID", studentID);
        cmd.Parameters.AddWithValue("@advisorID", advisorID);
        cmd.Parameters.AddWithValue("@date", DateTime.Now);
        cmd.Parameters.AddWithValue("@reason", txtReason.Text);
        cmd.Parameters.AddWithValue("@status", "Requested");

        cmd.ExecuteNonQuery();
    }

    LoadAppointments();
    ClearFields();
    MessageBox.Show("Appointment created successfully.");
}

// Update
private void btnUpdate_Click(object sender, EventArgs e)
```

```
{
    if (txtAppointmentID.Text == "" || txtStatus.Text == "")
    {
        MessageBox.Show("Please enter the appointment ID and the new status.");
        return;
    }

    if (!int.TryParse(txtAppointmentID.Text, out int appointmentID))
    {
        MessageBox.Show("Appointment ID must be a valid integer.");
        return;
    }

    string query = "UPDATE Appointments SET status = @status WHERE appointmentID = @appointmentID";

    using (SqlConnection conn = new SqlConnection(connectionString))
    using (SqlCommand cmd = new SqlCommand(query, conn))
    {
        conn.Open();
        cmd.Parameters.AddWithValue("@status", txtStatus.Text);
        cmd.Parameters.AddWithValue("@appointmentID", appointmentID);
        cmd.ExecuteNonQuery();
    }

    LoadAppointments();
    ClearFields();
    MessageBox.Show("Appointment updated successfully.");
}
```

```
// Delete
private void btnDelete_Click(object sender, EventArgs e)
{
    if (txtAppointmentID.Text == "")
    {
        MessageBox.Show("Please enter the appointment ID to delete.");
        return;
    }

    if (!int.TryParse(txtAppointmentID.Text, out int appointmentID))
    {
        MessageBox.Show("Appointment ID must be a valid integer.");
        return;
    }

    string query = "DELETE FROM Appointments WHERE appointmentID = @appointmentID";

    using (SqlConnection conn = new SqlConnection(connectionString))
    using (SqlCommand cmd = new SqlCommand(query, conn))
    {
        conn.Open();
        cmd.Parameters.AddWithValue("@appointmentID", appointmentID);
        cmd.ExecuteNonQuery();
    }
    LoadAppointments();
    ClearFields();
    MessageBox.Show("Appointment deleted successfully.");
```

```
}

// Helper method used by Create, Update and Delete
private void ClearFields()
{
    txtStudentID.Text = "";
    txtAdvisorID.Text = "";
    txtReason.Text = "";
    txtAppointmentID.Text = "";
    txtStatus.Text = "";
}

// Optional helper – dgvAppointments_CellClick
private void dgvAppointments_CellClick(object sender, DataGridViewCellEventArgs e)
// This method runs automatically when the user clicks a cell inside dgvAppointments.
{
    if (e.RowIndex >= 0)
    // Checks that the user clicked a real data row.
    // This prevents errors if the user clicks the column header row.
    {
        txtAppointmentID.Text = dgvAppointments.Rows[e.RowIndex].Cells["appointmentID"].Value.ToString();
        // Reads the appointmentID value from the selected row
        // and places it into the txtAppointmentID textbox.

        txtStatus.Text = dgvAppointments.Rows[e.RowIndex].Cells["status"].Value.ToString();
        // Reads the status value from the selected row
        // and places it into the txtStatus textbox.
    }
```

```
}
```

Part 4: DevOps & Configuration Note

Part 4: DevOps & Configuration Note I followed basic DevOps practices by keeping configuration in one central place. The connection string is stored in App.config under the <connectionStrings> section. All SQL scripts are saved in a folder called DatabaseScripts. The connection string is read once using ConfigurationManager and reused everywhere in the project. This makes future deployment or server changes much easier and safer.

Part 5: Personal Journal 4

Personal Journal 4 The biggest technical challenge was learning how to safely pass values with parameters and how to handle exceptions with try-catch. Switching to SQL Server made my project feel much more professional because data now persists after closing the program. I also learned that keeping the connection string in one central place is a basic DevOps practice that makes future deployment easier.

Instructor Note

This assignment directly follows Lab 8 and the Week 8 lecture on DevOps & deployment strategies (1/2). Students should use the same techniques shown in Chapter 8 and Lab 8. The focus is on a working, persistent database integrated into their Windows Forms project.